

Título: Documento de Requerimientos Detallados – Backend App de Cobros

Autor: ChatGPT

Fecha: Generado automáticamente

1. Introducción

Este documento detalla todos los requerimientos funcionales, no funcionales y técnicos necesarios para desarrollar un backend que procese pagos provenientes de Wispro, los almacene internamente, los compare contra facturas provenientes de Google Sheets y genere resultados contables consolidados. La implementación propuesta utiliza Node.js como lenguaje principal y MySQL como base de datos relacional.

2. Objetivo del Backend

El backend debe centralizar, procesar y relacionar los pagos obtenidos desde Wispro con las facturas registradas en Google Sheets para producir un reporte final contable, clasificando cada factura según coincidencias entre pagos y montos facturados.

Objetivos específicos:

- Obtener los pagos del API Wispro según un rango de fechas.
- Desglosar cada pago y relacionarlo con las facturas afectadas.
- Guardar la información internamente sin generar duplicados.
- Leer las facturas desde Google Sheets.
- Comparar pagos vs facturas y clasificarlas (exactas, parciales, sobrepagos, no encontradas).
- Crear un conjunto de resultados exportables.
- Registrar logs de errores y procesos relevantes.

3. Flujo General del Sistema

- 1) El usuario envía un rango de fechas al backend.
- 2) El backend consume la API de Wispro para obtener todos los pagos exitosos.
- 3) Se almacenan los pagos y su detalle en MySQL evitando duplicados.
- 4) Se obtiene desde Google Sheets la base de facturas.
- 5) El sistema normaliza números de factura y campos relevantes.

6) Se comparan los montos pagados contra el total de cada factura.

7) Se clasifica cada factura.

8) Se genera una salida final para Google Sheets o exportación.

9) Se registran logs del proceso.

4. Requerimientos Funcionales

4.1. Consulta de pagos Wispro

- Ruta API: GET /pagos?inicio=YYYY-MM-DD&fin;=YYYY-MM-DD
- Debe consultar Wispro paginando resultados.
- Filtrar únicamente pagos con estado "success".
- Para cada pago:
 - Obtener detalles ("payment_transactions").
 - Extraer documento (número de factura), monto aplicado, fecha y campos adicionales.
- Guardar:
 - Tabla pagos
 - Tabla pagos_detalle (pago ↔ factura)

4.2. Almacenamiento interno (MySQL)

Tablas requeridas:

A) pagos

- id_pago (PK)
- fecha_pago
- cliente
- banco
- transaction_code
- monto_total
- estado

B) pagos_detalle

- id_detalle (PK)
- id_pago (FK)
- numero_factura
- monto_aplicado
- otros metadatos necesarios

C) facturas_sheets

- numero_factura
- total_factura
- fecha_emision
- cliente

D) comparacion_resultados

- numero_factura
- total_pagado
- total_factura
- diferencia
- estado_clasificacion
- banco
- referencia
- cliente

4.3. Lectura de Google Sheets

- Ruta: GET /facturas/importar
- El sistema debe conectarse a la API de Google Sheets.
- Descargar hoja configurada.
- Guardar la información en facturas_sheets.
- Normalizar:

- Quitar espacios
- Eliminar ceros a la izquierda
- Unificar formatos de número de factura

4.4. Comparación de pagos vs facturas

- Ruta: POST /comparar
- Para cada factura detectada en pagos_detalle:
 - Sumar el total monto pagado.
 - Buscar factura en Sheets.
 - Clasificar:
 - * EXACTA: $\text{total_pagado} == \text{total_factura}$
 - * PARCIAL: $\text{total_pagado} < \text{total_factura}$
 - * SOBREPAGO: $\text{total_pagado} > \text{total_factura}$
 - * NO ENCONTRADA: no existe en Sheets
 - Insertar los resultados en comparacion_resultados.

4.5. Generación de salida final

- Ruta: GET /exportar
- Exportar resultados a JSON, Excel o a Google Sheets.
- Debe incluir solo información necesaria para contabilidad.

4.6. Logging

Registrar:

- Cantidad de pagos obtenidos
- Cantidad insertada
- Duplicados detectados
- Errores de API Wispro
- Errores Google Sheets

- Facturas inconsistentes

5. Requerimientos No Funcionales

- Escalabilidad: Able para manejar miles de pagos.
- No duplicación: Validaciones estrictas por transaction_code.
- Normalización estandarizada.
- Seguridad:
- Validación de API Key Wispro.
- Protección de credenciales de Google.
- Middleware de validación de solicitudes.

6. Arquitectura Propuesta

- Node.js con ExpressJS para el backend REST.
- MySQL para almacenamiento relacional.
- Sequelize/Prisma para ORM.
- Servicios separados:
- wispro.service.js
- sheets.service.js
- comparacion.service.js
- exportacion.service.js
- Controladores REST:
- pagos.controller.js
- facturas.controller.js
- comparacion.controller.js

7. Endpoints Detallados

GET /pagos

Descripción: Trae pagos desde Wispro.

Query: inicio, fin

Resultado: Pagos almacenados en MySQL.

POST /pagos/desglosar

Descripción: Procesa detalles de cada pago.

GET /facturas/importar

Descripción: Lee facturas del Google Sheets.

POST /comparar

Descripción: Compara pagos vs facturas y clasifica.

GET /exportar

Descripción: Exporta resultados.

8. Validaciones Clave

- Número de factura debe ser consistente en ambos sistemas.
- transaction_code debe ser único.
- Montos deben compararse con precisión decimal.
- Logs obligatorios en cada proceso.

9. Criterios de Aceptación

- El backend puede procesar un rango completo sin fallas.
- Ningún pago debe duplicarse.
- La comparación debe generar los 4 estados correctamente.
- Debe producir un archivo final listo para contabilidad.
- Debe manejar faltas de conexión a Wispro o Google Sheets con logs adecuados.

10. Conclusión

El backend descrito permite centralizar los pagos provenientes de Wispro, validar y relacionar esos pagos con sus respectivas facturas almacenadas en Google Sheets, y generar una salida estandarizada que facilita la gestión contable. Usando Node.js y MySQL se garantiza rendimiento, flexibilidad y facilidad de mantenimiento.